Drugi popravni test iz Algoritama i struktura podataka

Prvi praktični test održan 19.02.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test2_GrupaA_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Konverzija liste u matricu susedstva*

Napisati algoritam koji će vršiti direktnu konverziju liste susedstva u matricu susedstva. Algoritam kao ulazni parametar treba da prihvati dinamički niz čvorova, a treba da rezultuje kreiranom matricom.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***ConvertAdjacencyListToAdjacencyMatrix*** koja treba da sadrži implementaciju konverzije liste u matricu susedstva.
- U *Main* metodi obezbediti način za testiranje i proveru nove funkcionalnosti.

Zadatak 2 – *Svi karakteri prvog prisutni u drugom*

Napisati metodu koja prima dva stringa i proverava da li se **svi jedinstveni karakteri** iz prvog stringa pojavljuju barem jednom u drugom stringu (bez obzira na broj pojavljivanja).

Dodatni zahtevi za rešavanje zadatka:

- Koristiti Hash tabelu iz šablona za čuvanje broja pojavljivanja svakog karaktera.
- Implementaciju pisati u metodi: ***public static bool areAllCharsPresent(string str1, string str2)***
- Ignorisati razliku između malih i velikih slova.
- Ukoliko jedna reč ima više istih slova, dovoljno je postojanje samo jednog karaktera u drugoj reči. (Pogledaj primer 1)

Primer:

Input 1:	*str1 = „Aba“, str2 = „Balkon“*
Output 1:	*true (Karakteri „a“ i „b“ postoje u drugom stringu)*
Input 2:	*str1 = „Kod“, str2 = „Algoritam“*
Output 2:	*false (Karakeri „d“ i „k“ ne postoji u drugom stringu)*

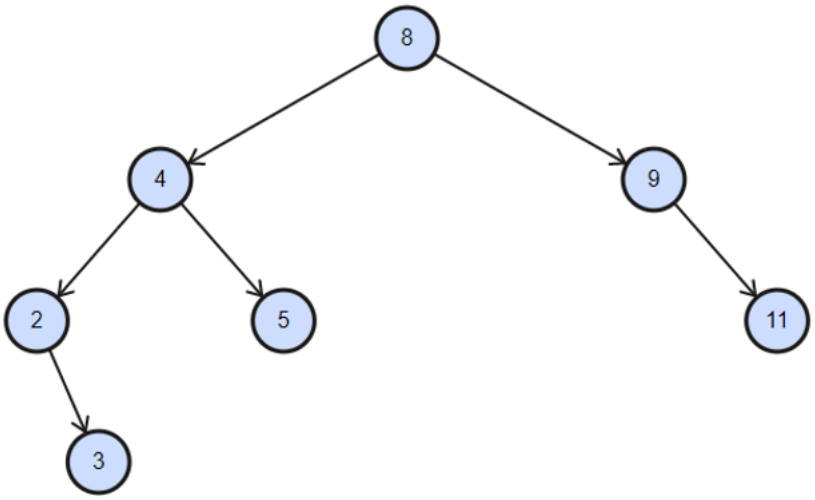
Zadatak 3 – *Provera "Putanja rasta" (Root-to-Leaf Max Path)*

U okviru klase BinarySearchTree napisati metodu koja proverava da li postoji putanja od korena do lista na kojoj je **svaki čvor strogo veći od zbira svih svojih predaka** na toj putanji.

Dodatni zahtevi za rešavanje zadatka:

1. Javna metoda: ***public bool existsSuperIncreasingPath()***
2. Privatna metoda: ***private bool checkSuperIncreasingPath(Node current, int currentSum)***.
3. Podrazumeva se da stablo ima barem dva čvora.
4. Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

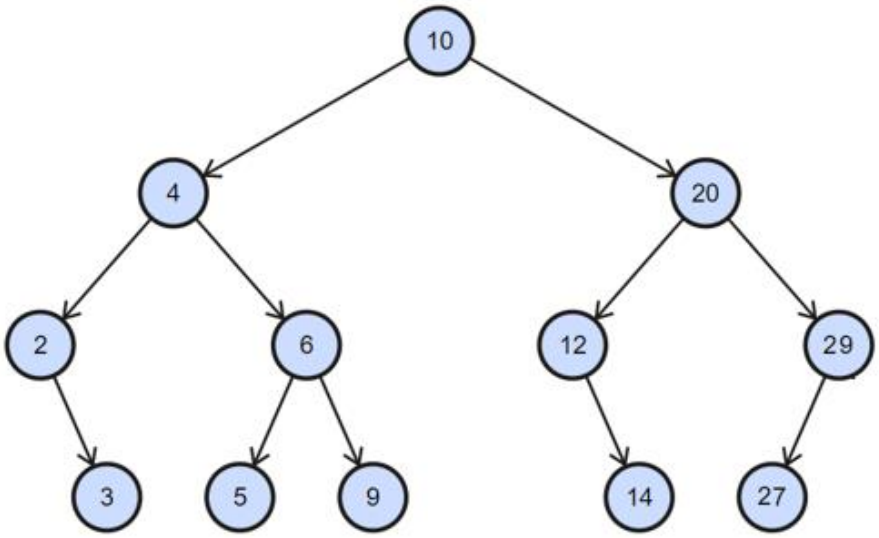
Primeri:



Slika 1 - Primer BST-a

Input 1: *BST sa slike 1*

Output 1: *false (11 nije veće od 9 + 8)*



Slika 2 - Primer BST-a

Input 2: *BST sa slike 2*

Output 2: *false (29 nije veće od 10 + 20)*